Teoria Współbieżności Zadanie domowe Lab 5

Oliwia Dyszewska

listopad 2025

1 Opis zadania

Dane są:

- Alfabet A, w którym każda litera oznacza akcję,
- Zestaw transakcji na zmiennych
- Słowo w oznaczające przykładowe wykonanie sekwencji akcji.

1.1 Zadanie (7 pkt.)

Napisz program, który:

1. Wyznacza relację zależności D (2 pkt.)
2. Wyznacza relację niezależności I (1 pkt.)
3. Wyznacza postać normalną Foaty FNF([w]) śladu [w] (2 pkt.)
4. Rysuje graf zależności w postaci minimalnej dla słowa w (2 pkt.)

2 Opis programu

Do implementacji programu użyłam języka Python, do rysowania grafu skorzystałam z programu Graphviz.

Kod programu znajduje się w pliku *program.py*. Do jego uruchomienia wymagane są:

- interpreter Python 3,
- biblioteka Pythona graphviz,
- zainstalowany w systemie program Graphviz.

Zakładając, że plik *case1.txt* znajduje się w tym samym katalogu co *program.py*, a jego struktura jest taka jak w poniższym przykładzie: *case1.txt*

```
(a) x := x + 1
(b) y := y + 2z
(c) x := 3x + z
(d) w := w + v
(e) z := y - z
(f) v := x + v
A = {a, b, c, d, e, f}
w = acdcfbbe
```

Uruchamiamy program następującą komendą w terminalu:

```
python program.py case1.txt
```

W pliku *program.py* znajdują się następujące funkcje:

- *split*
  Zwraca lewą i prawą stronę równania.

```
def split(expr):
    expr = expr.strip()
    if ':=' in expr:
        left, right = expr.split(':=', 1)
    elif '=' in expr:
        left, right = expr.split('=', 1)
    else:
        raise ValueError(f"Nie rozpoznano w: {expr}")
    return left.strip(), right.strip()
```

- *get_transactions*
  Linie z formatu jak w zadaniu ((a) x := x + 1)), zapisuje w postaci ułatwiającej liczenie relacji D i I ('a': 'R': 'x', 'W': 'x', R-odczyt. W- zapis.).

```
def get_transactions(file):
    text = file.read()
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    transactions = {}
    for line in lines:
        m = re.match(r'\(([A-Za-z])\)\s*(.+)$', line)
        if not m:
            continue  # ignorujemy np. linie "A = {...}" albo "w = ..."
        tr, expr = m.group(1), m.group(2)
        left, right = split(expr)
        R = set(re.findall(r'[A-Za-z]+', right))
        W = {left}
        transactions[tr] = {'R': R, 'W': W}
    return transactions
```

- *parse_input_file*
  Parsuje jeden plik zawierający: linie transakcji '(a) x := ...', linię 'A = ...', linię 'w = ...'. Zwraca (transactions, alphabet, word).

```
def parse_input_file(text):
    # transakcje
    transactions = get_transactions(io.StringIO(text))

    # alfabet
    a = re.search(r'A\s*=\s*\{([^}]*)\}', text)
    if a:
        letters = re.findall(r'[a-z]', a.group(1))
        alphabet = sorted(set(letters))
    else: # jeśli nie ma podanego alfabetu to można go wyciągnąć z transakcji
        alphabet = sorted(transactions.keys())

    # słowo
    w = re.search(r'\bw\s*=\s*([a-z]+)', text)
    if not w:
        raise ValueError("Nie znaleziono linii 'w = ...' w pliku case.")
    word = w.group(1)

    return transactions, alphabet, word
```

- *get_dependent_transactions*
  Wyznacza relację zależności D na alfabecie.

```
def get_dependent_transactions(transactions, alphabet):
    D = set()
    for p in alphabet:
        for q in alphabet:
            Wp, Rp = transactions[p]['W'], transactions[p]['R']
            Wq, Rq = transactions[q]['W'], transactions[q]['R']
            conflict = (
                p == q or
                (Wp & Wq) or
                (Wp & Rq) or
                (Rp & Wq)
            )
            if conflict:
                D.add((p, q))
    return D
```

- *get_dependencies*
  Zwraca słownik: litera -> zbiór liter, od których zależy (bez siebie).

```
def get_dependencies(dependent, alphabet):
    dep = {a: set() for a in alphabet}
    for a, b in dependent:
        if a != b:
            dep[a].add(b)
    return dep
```

- *reduction*
  Usuwa (u,v) jeśli istnieje ścieżka u->v przez co najmniej jeden węzeł.

```
def reduction(edges):
    l = defaultdict(list)
    for u, v in edges:
        l[u].append(v)

    reduced = set(edges)
    for (u, v) in list(edges):
        stack = [x for x in l[u] if x != v] # stos wierzchołków do odwiedzenia
        seen = set([u])                     # zbiór odwiedzonych wierzchołków
        reachable = False                   # czy dotarliśmy do v
        while stack:
            w = stack.pop()
            if w == v:
                reachable = True
                break
            if w in seen:
                continue
            seen.add(w)
            stack.extend(l[w])
        if reachable:
            reduced.discard((u, v))
    return reduced
```

- *get_FNF*
  Zwraca postać normalną Foaty FNF([w]) śladu [w].

```
def get_FNF(dependencies, word):
    n = len(word)
    edges = build_edges(word, dependencies)
    edges = reduction(edges)

    # Listy sąsiedztwa: wejściowe (in_adj) i wyjściowe (out_adj)
    out_list = defaultdict(set)
    in_list = defaultdict(set)
    for u, v in edges:
        out_list[u].add(v)
        in_list[v].add(u)

    # Zbiór jeszcze nie "użytych" pozycji w słowie
    remaining = set(range(1, n + 1))

    fnf = []

    # Warstwowe zdejmowanie wierzchołków bez poprzedników
    while remaining:
        # Pozycje bez krawędzi przychodzących z pozostałych wierzchołków
        layer_positions = sorted(
            i for i in remaining
            if not (in_list[i] & remaining)
        )

        # Litery w tej warstwie
        layer_letters = [word[i - 1] for i in layer_positions]
        fnf.append(tuple(sorted(layer_letters)))

        # Usuwamy wierzchołki tej warstwy z grafu
        for i in layer_positions:
            remaining.remove(i)
            for v in out_list[i]:
                in_list[v].discard(i)

    return fnf
```

- *build_edges*
  Zwraca krawędzie, i->j jeśli i<j oraz (word[i] zależy od word[j]) lub word[i]==word[j].

```
def build_edges(word, dependencies):
    edges = set()
    n = len(word)
    for i in range(n - 1):
        for j in range(i + 1, n):
            if word[j] in dependencies[word[i]] or word[i] == word[j]:
                edges.add((i + 1, j + 1))
    return edges
```

2.1 Działanie programu:

1. Program najpierw otwiera plik podany w argumencie oraz przetwarza dane znajdujące się w nim.
2. Następnie wyznacza relację zależności i niezależności.
3. Wyznacza FNF
4. Wyznacza graf w postaci DOT
5. Rysuje graf za pomocą Graphviz oraz zapisuje go do pliku graph.png

```
if __name__ == '__main__':
    if len(sys.argv) == 2:
        filename = sys.argv[1]
        try:
            with open(filename, encoding='utf8') as f:
                text = f.read()
        except IOError:
            print(f"Nie udało się otworzyć pliku: {filename}")
            sys.exit(0)

        transactions, alphabet, word = parse_input_file(text)
    else:
        print("Nieprawidłowa liczba argumentów! \n Prawidłowe uruchomienie: python 
            program.py <plik>")
        sys.exit(0)

    # Wyznaczenie relacji zależności i niezależności
    sigma2 = [(ai, aj) for aj in alphabet for ai in alphabet]
    dependent = get_dependent_transactions(transactions, alphabet)
    independent = sorted(list(set(sigma2) - dependent))     # I = (sigma)^2 \ D
    dependent = sorted(list(dependent))

    print("\nRelacja zależności:\n D = {", end = "")
    for i in range(len(dependent)):
        if i > 0: print(",", end = "")
        print(f"({dependent[i][0]},{dependent[i][1]})", end = "")
    print("}")

    print("\nRelacja niezależności:\nI = {", end = "")
    for i in range(len(independent)):
        if i > 0: print(",", end = "")
        print(f"({independent[i][0]},{independent[i][1]})", end = "")
    print("}")

    # Wyznaczenie FNF
    dependencies = get_dependencies(dependent, alphabet)
    fnf = get_FNF(dependencies, word)
    print("\nFNF:\nFNF([w]) = ", end = "")
    for f in fnf:
        print("(", end = "")
        for transaction in sorted(f):
            print(transaction, end = "")
        print(")", end = "")
    print()

    # Wznaczenie grafu w postaci DOT
    graph_edges = reduction(build_edges(word, dependencies))
    print("\nGraf w postaci dot:\ndigraph g{")
    for edge in sorted(graph_edges, key=lambda edge: (edge[1], edge[0])):
```

```
        u, v = edge
        print(f"    {u} -> {v};")
    for i, ch in enumerate(word, start=1):
        print(f'    {i} [label="{ch}"];')
    print("}\n")

    # Rysowanie grafu za pomocą Graphviz
    dot = Digraph("g")
    for i, ch in enumerate(word, start=1):
        dot.node(str(i), label=ch)

    for u, v in sorted(graph_edges, key=lambda e: (e[1], e[0])):
        dot.edge(str(u), str(v))

    try:
        dot.render("graph", format="png", cleanup=True)
        print("Graf zapisano do pliku graph.png")
    except Exception as e:
        print(e)
```

3 Wyniki działania dla przykładowych danych

Program w terminalu zwróci kolejno:

- relacje zależności,
- relacje niezależności,
- FNF,
- graf w postaci DOT,
- informację, że graf zapisano do pliku graph.png

Dodatkowo w katalogu, w którym znajduje się program powstanie/nadpisze się plik graph.png. Dla lepszej czytelności w poniższych wyniki relacji zależności i niezależności, które w terminalu wyświetlają się w jednej lini, podzieliłam na kilka lini.

3.1 Wyniki dla case1.txt

3.1.1 Zawartość pliku:

```
(a) x := x + 1
(b) y := y + 2z
(c) x := 3x + z
(d) w := w + v
(e) z := y - z
(f) v := x + v
A = {a, b, c, d, e, f}
w = acdcfbbe
```

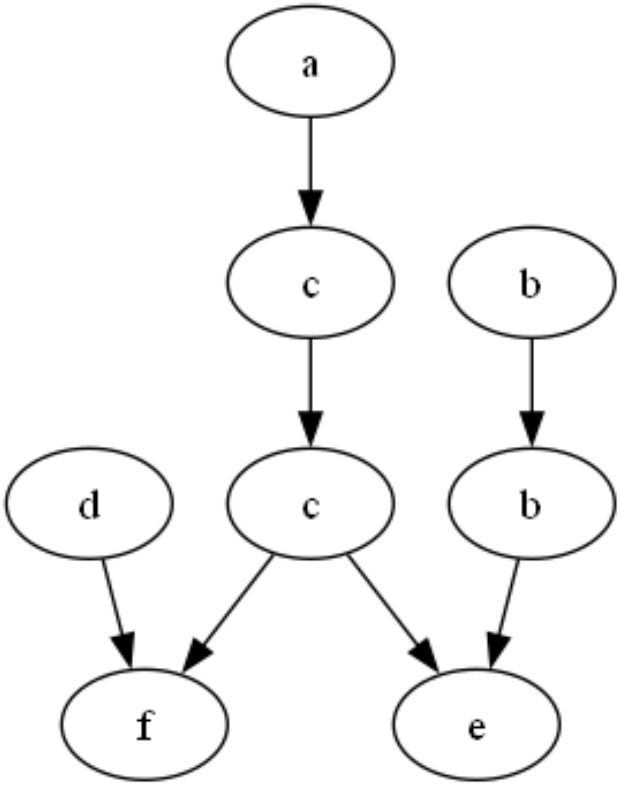
3.1.2 Uruchomienie programu:

```
python program.py case1.txt
```

3.1.3 Wynik na terminalu:

```
Relacja zależności:
D = {(a,a),(a,c),(a,f),(b,b),(b,e),(c,a),(c,c),(c,e),(c,f),(d,d),(d,f),(e,b),(e,c),
      (e,e),(f,a),(f,c),(f,d),(f,f)}

Relacja niezależności:
I = {(a,b),(a,d),(a,e),(b,a),(b,c),(b,d),(b,f),(c,b),(c,d),(d,a),(d,b),(d,c),(d,e),
    (e,a),(e,d),(e,f),(f,b),(f,e)}

FNF:
FNF([w]) = (abd)(bc)(c)(ef)

Graf w postaci dot:
digraph g{
    1 -> 2;
    2 -> 4;
    3 -> 5;
    4 -> 5;
    6 -> 7;
    4 -> 8;
    7 -> 8;
    1 [label="a"];
    2 [label="c"];
    3 [label="d"];
    4 [label="c"];
    5 [label="f"];
    6 [label="b"];
    7 [label="b"];
    8 [label="e"];
}

Graf zapisano do pliku graph.png
```

3.1.4 Rysunek grafu:



Rysunek 1: graph.png

3.2 Wyniki dla case2.txt

3.2.1 Zawartość pliku:

```
(a) x := x + y
(b) y := z - v
(c) z := v * x
(d) v := x + 2y
(e) x := 3y + 2x
(f) v := v - 2z
A = {a,b,c,d,e,f}
w = afaeffbcd
```

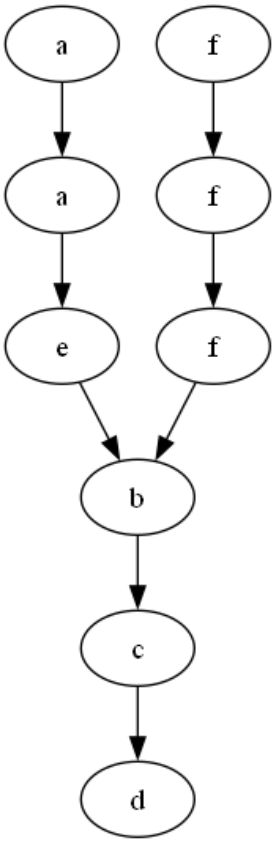
3.2.2 Uruchomienie programu:

```
python program.py case2.txt
```

3.2.3 Wynik na terminalu:

```
Relacja zależności:
D = {(a,a),(a,b),(a,c),(a,d),(a,e),(b,a),(b,b),(b,c),(b,d),(b,e),(b,f),(c,a),(c,b),
      (c,c),(c,d),(c,e),(c,f),(d,a),(d,b),(d,c),(d,d),(d,e),(d,f),(e,a),(e,b),(e,c),
      (e,d),(e,e),(f,b),(f,c),(f,d),(f,f)}

Relacja niezależności:
I = {(a,f),(e,f),(f,a),(f,e)}

FNF:
FNF([w]) = (af)(af)(ef)(b)(c)(d)

Graf w postaci dot:
digraph g{
    1 -> 3;
    3 -> 4;
    2 -> 5;
    5 -> 6;
    4 -> 7;
    6 -> 7;
    7 -> 8;
    8 -> 9;
    1 [label="a"];
    2 [label="f"];
    3 [label="a"];
    4 [label="e"];
    5 [label="f"];
    6 [label="f"];
    7 [label="b"];
    8 [label="c"];
    9 [label="d"];
}

Graf zapisano do pliku graph.png
```

3.2.4 Rysunek grafu:



Rysunek 2: graph.png

3.3 Wyniki dla case3.txt

3.3.1 Zawartość pliku:

```
(a) x := x + y
(b) y := y + 2z
(c) x := 3x + z
(d) z := y      z
A = {a, b, c, d}
w = baadcb
```

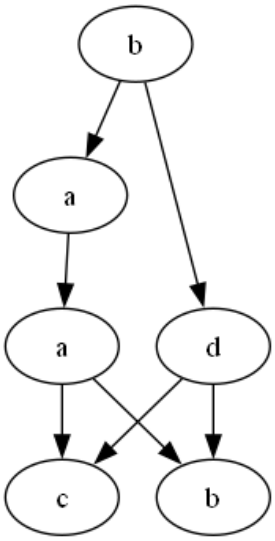
3.3.2 Uruchomienie programu:

```
python program.py case3.txt
```

3.3.3 Wynik na terminalu:

```
Relacja zależności:
D = {(a,a),(a,b),(a,c),(b,a),(b,b),(b,d),(c,a),(c,c),(c,d),(d,b),(d,c),(d,d)}

Relacja niezależności:
I = {(a,d),(b,c),(c,b),(d,a)}

FNF:
FNF([w]) = (b)(ad)(a)(bc)

Graf w postaci dot:
digraph g{
    1 -> 2;
    2 -> 3;
    1 -> 4;
    3 -> 5;
    4 -> 5;
    3 -> 6;
    4 -> 6;
    1 [label="b"];
    2 [label="a"];
    3 [label="a"];
    4 [label="d"];
    5 [label="c"];
    6 [label="b"];
}

Graf zapisano do pliku graph.png
```

3.3.4 Rysunek grafu:



Rysunek 3: graph.png